

An open door without a floor

Station Rescue / M-07 | Unity 6000.6.4f1 | 2026-10-03 Development post-mortem - draft for student review

The problem and expected behaviour

During a playtest, the player restored power, waited for the airlock to open, and drove the robot through it without pressing Interact. The robot fell below the station instead of staying on the visible docking platform.

This game deliberately uses manual boarding. Reaching an open airlock should not complete the mission. However, the player should be able to walk onto its platform, remain supported, and return to the boarding marker to press E or click the action button.

Narrowing down the cause

First, the boarding state and door-collision rules were inspected. One candidate explanation was premature removal of the door colliders. The code disables those colliders only when `IsOpen` is true, and the report concerned a fully open door. Keeping that closed-door barrier forever would hide the symptom while contradicting the visible open passage.

A second possibility was that walking through the exit should trigger automatic boarding. That was rejected as a design fix: the agreed interaction requires an explicit action, and the task state correctly stayed `Installed` without E. Automatic completion would change the rule rather than repair the unsafe space.

The scene builder then revealed the physical cause. `LandingPad` was created as a visible cube, but the shape helper defaults to a non-solid object. Its renderer existed while its Collider had been removed. Opening the hatch therefore exposed a platform with no physical support, followed by an unguarded outer edge.

Evidence from a failing regression

A new check powered dock A, placed the robot before the airlock, and drove forward for three seconds with the real `PlayerMotor` and `Physics.Simulate`, without interacting. On the old saved scene, the robot reached `y = -26.59`; the support assertion failed and Unity exited with code 1.

Why earlier checks missed it

The previous 293 Editor checks and standalone tests covered powering, reaching a boarding marker, and explicit boarding. They did not continue through the doorway without E. Their success was evidence for those paths, not proof that all newly reachable space was safe.

Source: `StationBuilder.Dock`, `DockGate.Apply` and the pre-fix log `Builds/dock-boundary-before-fix.log`.

Repair the space, preserve the rule

The same problem, followed through implementation and verification

The fix

LandingPad now retains a solid BoxCollider. Side rails and an end rail also retain colliders, so an open airlock leads to a supported, bounded platform. These objects are part of the repeatable scene builder, preventing the fix from disappearing when the generated scene is upgraded again.

Boarding remains manual. The hatch animation still opens normally; walking through it does not change the mission to Complete. The player returns to the visible marker and chooses E or the mouse action button to board.

Verification and a test-runner correction

The new Editor regression runs against all three selected docks. It drives forward and sideways after opening, checks that the robot stays above the platform, checks that rails limit its motion, and verifies that no completion occurs without interaction. The same support assertion that failed before the fix now passes; observed robot height was approximately 0.55-0.56.

The first Windows run passed the platform and rail checks but failed the return route at dock B. The test navigator stopped steering sideways too late, leaving residual speed that carried it beyond the centreline toward a doorpost. It was corrected to use the existing braking-aware waypoint control, then the actual keyboard route was rerun. This failure was kept separate from the platform defect.

The final Editor report contains 317 passed checks. The Windows report contains 176 passed checks, including walking past each of A/B/C without E, testing the side rail, walking back, and manually boarding. It also preserves the full WASD delivery mission and the existing interaction, pause, language and restart checks.

What will change in the next design brief

1. Describe the contract between appearance and physics: a visible route needs physical support, or an explicit intentional-fall rule with recovery.
2. Treat state transitions as changes to reachable space. Test the area behind an opening door, not just whether the door animation or task flag is correct.
3. Write acceptance cases as precondition -> player input -> expected result. Include no input, wrong input, repeated input, movement beyond a goal, pause and restart.
4. Preserve the chosen interaction rule while repairing the world. A shortcut such as automatic boarding can conceal a defect and change the player's experience.

Scope and provenance

Coding, test execution and this draft were assisted by AI. The problem came from the user's actual playtest; diagnosis and measurements come from the repository and recorded runs. The student should review the draft, add their own understanding and follow the course's AI-use rules before submission.

Reports: docs/station-v2_1-validation.json; docs/screenshots/station-v2_1/standalone-validation.json. This draft discusses one observed development problem and is limited to two pages.